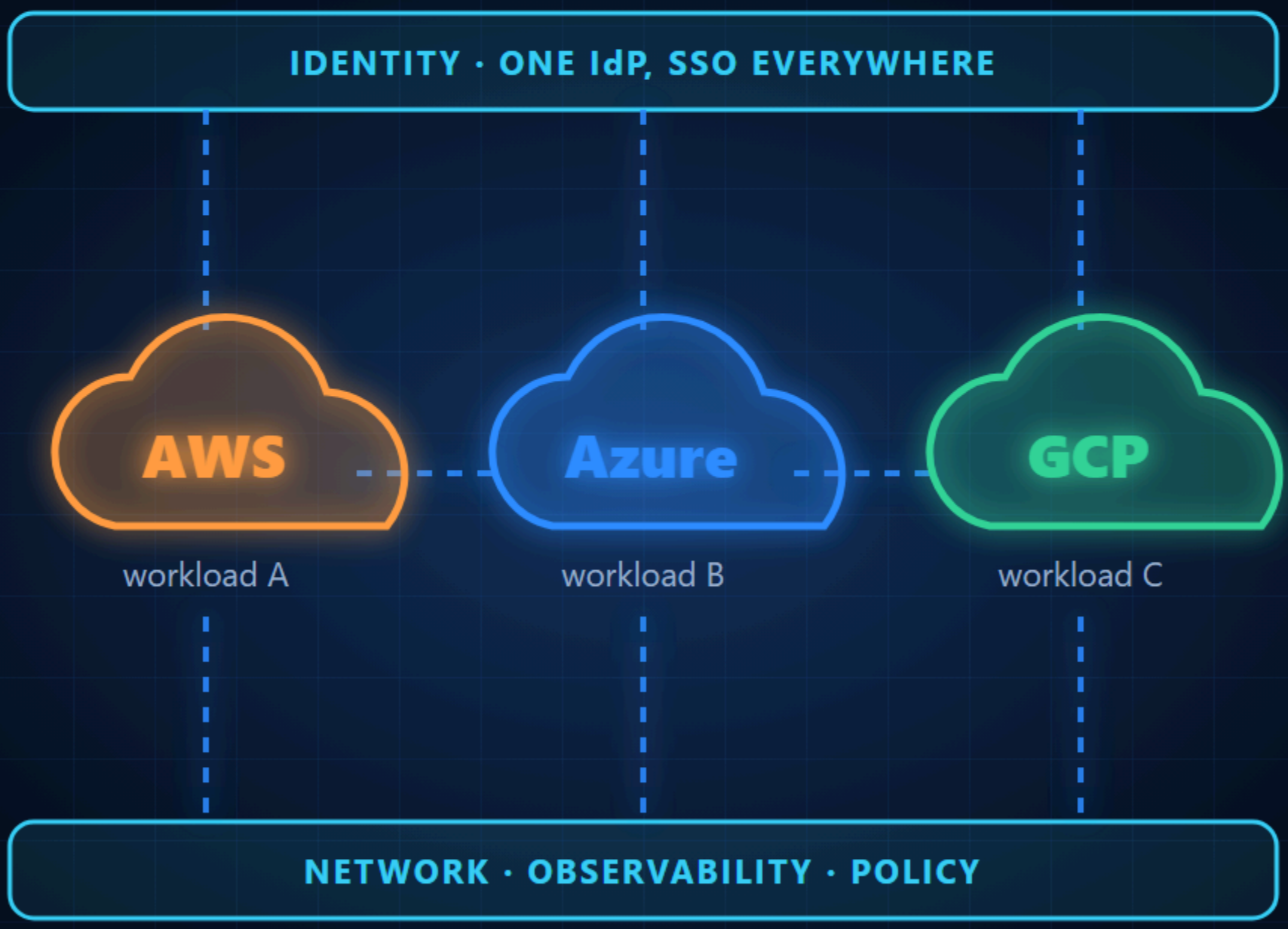
DAY 1 · MULTI-CLOUD ARCHITECTURE

3 clouds.
1 architecture.
Or 3 messes?



DEJI AJISHE · DevOps | Multi-Cloud

The layers that matter →

MULTI-CLOUD ARCHITECTURE 101

AWS + Azure + GCP: what does a real multi-cloud architecture look like?

Not three copies of everything. Workloads are placed per cloud. Shared layers run across all of them.

① IDENTITY

One identity provider, federated SSO into every cloud. No separate user lists.

AWS	Azure	GCP
Workload A	Workload B	Workload C
Its data	Its data	Its data

Place each workload where it fits best, and keep its data close to it

② NETWORK

Private connectivity, non-overlapping CIDRs, and a clear view of where egress costs money.

③ OBSERVABILITY

One telemetry pipeline (e.g. OpenTelemetry) feeding one place to look.

④ DELIVERY & POLICY

One CI/CD workflow and one policy/tagging standard, applied to every cloud.

WHAT & WHY

What multi-cloud architecture actually is

Multi-cloud = running workloads on two or more public clouds (AWS, Azure, GCP) **by design**, with shared identity, networking, delivery and operations standards across them.

Not the same as hybrid

Hybrid = on-premises + cloud. Multi-cloud = several public clouds. Many estates are both.

Not the same as multi-region

Multi-region = one cloud, several regions. Often the simpler route to high availability.

Why organizations do it

→ **Regulation and data residency:** data must live in a specific country or provider

→ **Best-fit services:** one cloud's data, AI or identity service is genuinely needed

→ **Mergers and acquisitions:** the acquired company runs on another cloud

→ **Vendor-risk policy:** limiting dependence on a single provider

When NOT to: "for resilience" alone. Multi-region in one cloud is usually cheaper and simpler.

REFERENCE ARCHITECTURE

Shared layers across, workloads placed per cloud

Identity: one IdP → SSO into AWS, Azure, GCP · workload identity, no static keys

Delivery: Git → CI (plan/test/scan) → Terraform apply → GitOps sync to clusters

AWS
Landing zone (Organizations)
VPC 10.10.0.0/16
EKS · workload A + data

Azure
Landing zone (mgmt groups)
VNet 10.20.0.0/16
AKS · workload B + data

GCP
Landing zone (folders)
VPC subnets 10.30.0.0/16
GKE · workload C + data

Network: private links (VPN / dedicated interconnect) · non-overlapping CIDRs · egress controls

Observability: OpenTelemetry Collector in each cloud → one backend · SLOs

Governance: one tagging standard · policy as code · budgets · tested DR

THE PROCESS

Building it: 8 steps, in order

01 **Define drivers and placement:** write down why each workload goes to each cloud

02 **Federate identity:** one IdP, SSO into every cloud, groups mapped to roles

03 **Plan IP space and connectivity:** non-overlapping CIDRs, private links

04 **Build a landing zone per cloud:** account structure, logging, guardrails

05 **Orchestrate infrastructure as code:** Terraform repo + pipeline with OIDC

06 **Standardize the workload platform:** Kubernetes + GitOps on every cloud

07 **Unify observability:** one telemetry pipeline, shared SLOs and alerts

08 **Govern and test:** tags, policy, budgets, and real failover drills

> **Order matters:** identity and IP planning are the hardest to retrofit. Do them before the first workload lands.

STEPS 1–2 · PLACEMENT & IDENTITY

Decide placement, then federate identity

Placement: score every workload

	Ask
Data	Where must the data live? What does it talk to most?
Services	Does it need a service only one cloud offers?
Latency	Will it make chatty calls across clouds? Keep them together.
Skills	Can on-call support it in that cloud at 2 AM?

Identity: one IdP, three clouds

```
# People: IdP (e.g. Entra ID / Okta) via SAML + SCIM
AWS   → IAM Identity Center   # permission sets per group
Azure → Entra ID              # RBAC on mgmt groups/subscriptions
GCP   → Workforce Identity Federation or Cloud Identity SSO

# Engineers sign in with the same identity everywhere
aws sso login --profile prod
az login
gcloud auth login
```

STEP 3 · NETWORK

One IP plan, private connectivity

```
# Agree before anything is created; never reuse a range
on-prem  10.0.0.0/16   aws  10.10.0.0/16
azure    10.20.0.0/16  gcp  10.30.0.0/16   reserved 10.40.0.0/13

# AWS
aws ec2 create-vpc --cidr-block 10.10.0.0/16
# Azure
az network vnet create -g rg-net -n vnet-prod \
  --address-prefixes 10.20.0.0/16 -l eastus
# GCP: custom-mode VPC, CIDRs live on subnets
gcloud compute networks create vpc-prod --subnet-mode=custom
gcloud compute networks subnets create snet-prod \
  --network=vpc-prod --region=us-central1 --range=10.30.0.0/20
```

	AWS	Azure	GCP
VPN	Site-to-Site VPN	VPN Gateway	Cloud VPN (HA VPN)
Dedicated	Direct Connect	ExpressRoute	Cloud Interconnect

STEP 5 · INFRASTRUCTURE ORCHESTRATION

One pipeline, three clouds, zero static keys

```yaml
# GitHub Actions: short-lived credentials via OIDC
permissions:
  id-token: write        # required for OIDC
  contents: read
steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: ${{ vars.AWS_ROLE_ARN }}
      aws-region: us-east-1
  - uses: azure/login@v2
    with:
      client-id: ${{ vars.AZ_CLIENT_ID }}
      tenant-id: ${{ vars.AZ_TENANT_ID }}
      subscription-id: ${{ vars.AZ_SUB_ID }}
  - uses: google-github-actions/auth@v2
    with:
      workload_identity_provider: ${{ vars.GCP_WIF }}
      service_account: ${{ vars.GCP_SA }}
  - run: terraform init && terraform plan -out=tfplan
  - run: terraform apply tfplan   # after review/approval
```

> **Rule:** one root config per cloud and environment, each with remote, locked state. Plan on every PR; apply only after review.

STEP 6 · WORKLOAD ORCHESTRATION

Same Kubernetes workflow on EKS, AKS and GKE

```
# Connect to each managed cluster
aws eks update-kubeconfig --name prod --region us-east-1
az aks get-credentials -g rg-prod -n prod
gcloud container clusters get-credentials prod --region us-central1
kubectl config get-contexts
```

```
# GitOps: one Argo CD ApplicationSet deploys to every registered cluster
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: { name: payments }
spec:
  generators:
    - clusters: {}            # every cluster Argo CD knows
  template:
    metadata: { name: 'payments-{{name}}' }
    spec:
      project: default
      source: { repoURL: 'https://github.com/acme/apps', path: payments }
      destination: { server: '{{server}}', namespace: payments }
```

> **Portable, not identical:** storage classes, load balancers, ingress and workload identity still differ per cloud.

STEPS 7–8 · OBSERVE & GOVERN

One place to look, one set of rules

Observability

→ **OpenTelemetry Collector** in every cluster, exporting to one backend

→ **Standard resource attributes** (cloud.provider, cloud.region, service.name) so one query covers all clouds

→ **Shared SLOs** (latency, errors) per service, with alerts on SLO burn, not CPU

Governance

→ **One tagging standard** with lowercase keys and values (GCP labels require it): owner, env, cost-center

→ **Policy as code** in CI: scan Terraform plans before apply

→ **Budgets and egress alerts** per cloud, reviewed monthly

→ **DR drills:** fail a dependency on purpose and measure recovery against your RTO/RPO

PITFALLS

8 mistakes that make multi-cloud fail

- → **Overlapping CIDRs:** peering and VPNs become impossible without renumbering
- → **Chatty cross-cloud calls:** latency and egress fees on every request
- → **Identity sprawl:** separate users per cloud, and leavers keep access
- → **Static cloud keys in CI:** long-lived secrets that leak; use OIDC
- → **Lowest-common-denominator abstraction:** losing each cloud's best services
- → **Three dashboards:** slow incident diagnosis across boundaries
- → **Untested failover:** a DR plan nobody has run is a hope, not a plan
- → **No cost owner:** egress and idle resources grow quietly

NEED-TO-KNOWS

What every professional should know

- → Multi-cloud does **not** automatically mean high availability
- → AWS subnets live in **one AZ**; Azure subnets **span zones**; GCP VPCs are **global**, subnets regional
- → Data **leaving** a cloud is usually billed; data coming in usually isn't
- → Azure resources always sit in a **resource group**; GCP resources in a **project**
- → OIDC federation lets CI deploy to **all three clouds** without stored keys
- → Kubernetes standardizes the **workload API**, not the cloud underneath
- → Terraform gives one **workflow**; resources stay **cloud-specific**
- → Write placement decisions down (ADRs). Future you will ask "why is this here?"

CHECKLIST

Multi-cloud readiness checklist

- ☐ Documented business driver for each cloud and workload
- ☐ One IdP with SSO everywhere; group-based roles; break-glass documented
- ☐ Written, non-overlapping IP plan; private connectivity in place
- ☐ Landing zone per cloud: accounts/subscriptions/projects, logging, guardrails
- ☐ Terraform per cloud and env; remote locked state; plan on every PR
- ☐ CI deploys with OIDC; no long-lived cloud keys anywhere
- ☐ Kubernetes + GitOps workflow identical across clusters
- ☐ One telemetry pipeline; SLOs and alerts per service
- ☐ Tagging standard, policy as code, budgets and egress alerts
- ☐ Failover tested in the last 90 days

FURTHER READING

Go deeper: official references

Google Cloud: Hybrid and multicloud architecture patterns
cloud.google.com/architecture/hybrid-multicloud-patterns

AWS: What is IAM Identity Center?
docs.aws.amazon.com/singlesignon/latest/userguide/what-is.html

GitHub: Security hardening with OpenID Connect
docs.github.com/en/actions/security-for-github-actions/security-hardening-your-deployments/about-security-hardening-with-openid-connect

Argo CD: ApplicationSet controller
argo-cd.readthedocs.io/en/stable/operator-manual/applicationset/

OpenTelemetry: Collector and cloud resource conventions
opentelemetry.io/docs/collector · opentelemetry.io/docs/specs/semconv/resource/cloud/

Docs change. Always check the current version before you design.

YOUR TURN

What would you place in each cloud, and why?

Share your design in the comments. I read and reply to every one.

Follow Deji Ajishe

Day 1 of my multi-cloud DevOps series

♻️ Repost to help an engineer on your team

linkedin.com/in/deji-ajishe-64a35722a